

実装差分・アーキテクチャ説明書

対象実装：2026年8月10日時点のデプロイ済みバージョン

対照した仕様書：「にゃんこ秘書AI 要件定義書 v3.2」（共通基盤）／「THE CONCIERGE 要件定義書 v1.0」（ブランド・画面）

先に押さえるべき4点

1. **Dify** を採用せず、**FastAPI** から **Claude** を直接呼んでいます。仕様書 v3.1／v3.2 の中核だった会話基盤 Dify を挟まず、v3.0 相当の構成に戻しました。仕様書 §17-2 が「撤退時の代替実装」として用意していた形を、最初から採っています。
2. **OAuthトークン**をサーバー側（**Supabase**）に保存しています。仕様書の設計制約「トークンはデバイスのセキュアストレージに保存し、サーバーに保存しないこと」から外れる、意図的な変更です。理由と代替策は後述します。
3. **朝のブリーフィングとリマインダー**は、サーバーからの**プッシュ通知**で届きます。仕様書は Cron サービスを前提にしていましたが、外部Cronは別課金になるため、常時起動しているバックエンド内で5分おきに判定する方式にしています。iPhone は「ホーム画面に追加」が受信条件です。
4. **仕様書に無い機能を4つ追加しています**。候補枠の仮押さえ、1プロバイダ内での複数カレンダー指定、重複予定の統合表示、予定検索ツールです。

いま動いている構成

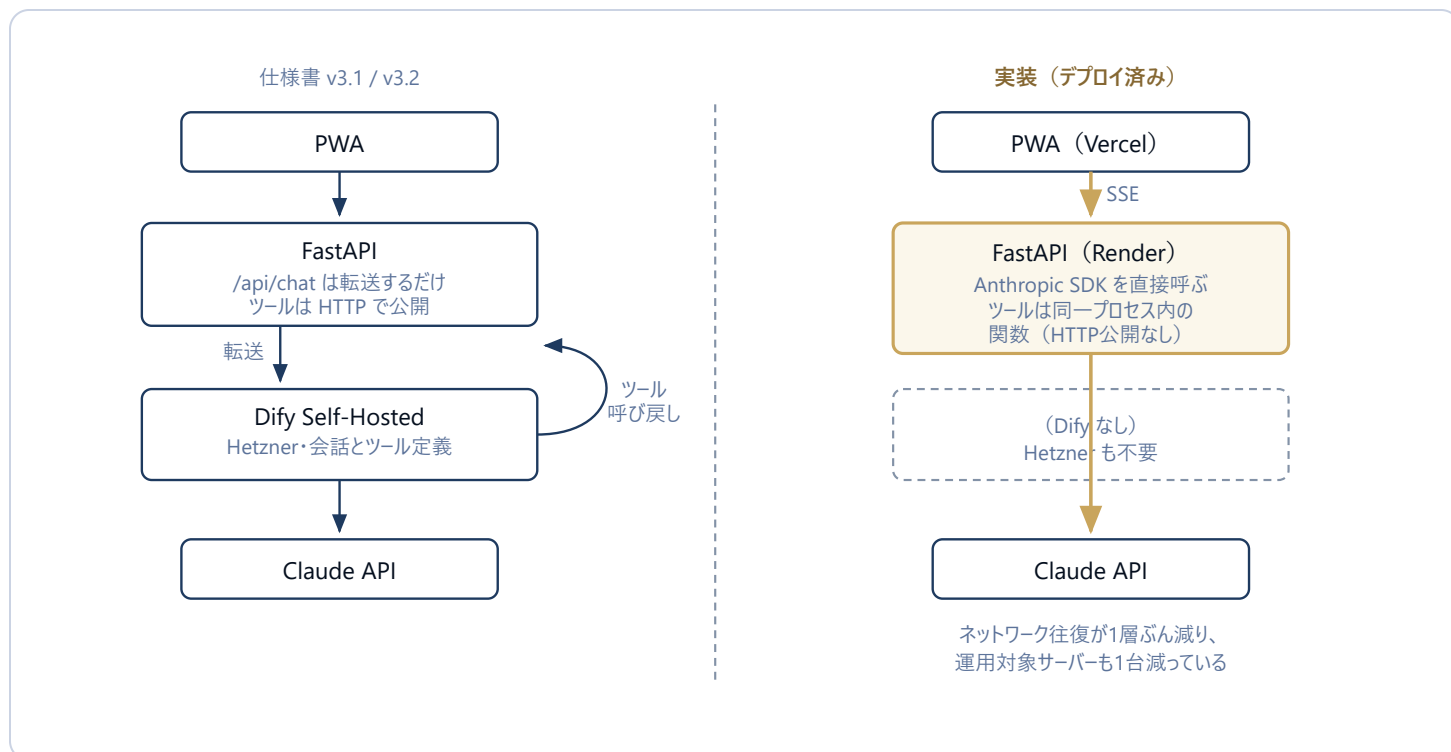


図1 仕様書の構成では、Claude がツールを使うたびに Dify から FastAPI へネットワーク往復が発生し、Dify 用のサーバー（Hetzner）も別途必要でした。実装ではその層ごとに取り除いています。

失ったものもあります。Dify の管理画面でプロンプトやツール定義を編集する運用、会話履歴の二重保存、そして「会話層だけ差し替えられる」撤退のしやすさです。現在プロンプトは `app/routers/chat.py` に、ツール定義は `app/tools.py` に直接書かれており、変更にはデプロイが必要です。

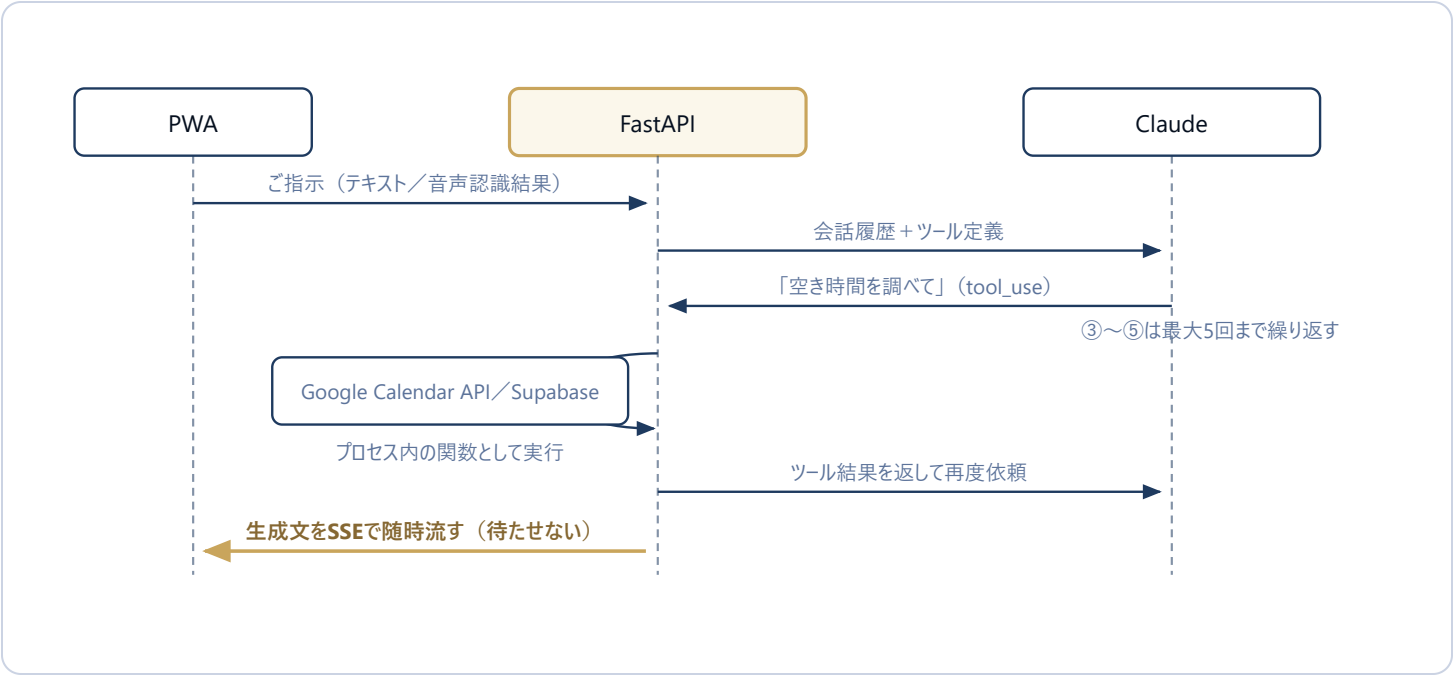


図2 チャット1往復の内部。Claude がツールを要求するたびに FastAPI が同一プロセス内で実行し、結果を返して続きを生成させます。文章は完成を待たず、生成しながら画面へ流しています。

デプロイ先

層	仕様書	実装
フロント (PWA)	Vercel (THE CONCIERGE §11-1)	Vercel
バックエンド	Render / Railway の無料枠	Render Starter (\$7/月)。無料枠は15分で停止し初回応答が遅いため変更
会話基盤	Dify Self-Hosted (Hetzner CX21・¥1,000/月)	なし
DB	Supabase	Supabase (同じ)
モデル	Claude Sonnet 4.6	claude-sonnet-5

トークンの置き場所を変えています

仕様書の制約から外れている箇所

両仕様書の「重要な設計制約」には「OAuth トークンはデバイスのセキュアストレージに保存し、サーバーに保存しないこと」とあります。実装では、Google から受け取ったアクセストークンとリフレッシュトークンを Supabase の `oauth_tokens` テーブルに保存しています。

変更した理由は2つあります。ひとつは、仕様書が想定していた Keychain／Keystore が PWA には存在しないこと。ブラウザで使える保管場所は localStorage 程度で、そこにリフレッシュトークンを置くほうが危険です。もうひとつは、朝のブリーフィングのようにユーザーが端末を開いていないときに動く処理を将来入れるなら、サーバーがトークンを持っていない限り実現できないことです。

そのうえで、次の形にしています。

- OAuth はサーバー側で完結する方式（認可コードフロー）にし、クライアントシークレットをブラウザに一切渡していません
- ブラウザが持つのは自前発行のセッショントークン（JWT・有効期限30日）だけで、これがあってもカレンダーには直接アクセスできません
- 全テーブルで Row Level Security を有効化済み

ただし、RLSは現時点では保険にとどまります

バックエンドは service_role キーで接続しているため RLS は実質バイパスされます。実際のユーザー分離は、全クエリに `user_id` の条件を必ず付けるというアプリ側の実装で担保しています。RLS が実効的な防御になるのは、将来フロントから Supabase を直接呼ぶ経路を作った場合です。仕様書 §22 のチェックリスト「全テーブルで RLS を有効化」は形式上満たしていますが、実態はこの通りである点をご認識ください。

なお、仕様書には**アプリ内のユーザー識別の設計そのものがありません**（単一ユーザー前提の記述になっています）。実装では `users`／`user_identities`／`oauth_tokens` の3テーブルを追加し、同じメールアドレスの Google と Outlook を1人のユーザーに統合できる構造にしました。これは以前ご相談した「§10-4 マルチユーザー対応」の内容ですが、仕様書側にはまだ反映されていません（Drive上の要件定義書の最終更新は2026年5月11日のままです）。

仕様書との差分一覧

変更

仕様と違う作りにした

追加

仕様に無いものを足した

未実装

仕様にあるが入っていない

Claude が使えるツール

ツール	区分	内容
<code>get_free_slots</code>	—	仕様どおり。Google と Outlook を並列取得。3件見つけ次第打ち切るため、どこまで調べたかを結果に含めています
<code>create_event</code>	変更	選択された 全登録先カレンダーに同時登録 するため、戻り値が単数から配列になっています
<code>reschedule_event</code>	変更	同じ予定が複数カレンダーにある場合、 全コピーを同時に動かします
<code>judge_memo_importance</code>	変更	キーワード一致による判定 です。仕様書の制約「毎回 Claude に判断させること」を満たしていません（下記参照）
<code>find_events</code>	追加	既存予定を検索して <code>event_id</code> を得るツール。仕様書 UC-5 は「find_event → reschedule_event」と書いていましたが §17 のツール一覧に無く、これが無いと予定変更が成立しないため追加しました
<code>create_task</code>	追加	会話からタスクを登録。時刻を伴う予定は <code>create_event</code> 、期限だけのやることは <code>create_task</code> と使い分けさせています
<code>get_travel_time</code>	未実装	Phase 2 の予定どおり未着手

メモ重要度判定について

仕様書は「メモ全文を Claude に判定させる」と定めていますが、実装は「持っていく・準備・印刷・提出・締め切り」など**7語のキーワード一致**で判定しています。判定のたびに Claude を呼ぶコストと遅延を避けた形ですが、仕様どおりの精度は出ません。Phase 1 の検証で「重要判定が当たらない」という声が出たら、ここを差し替える想定です。

データ設計（Supabase）

テーブル／カラム	区分	内容
events / tasks	—	仕様書 §16 どり
messages.dify_session_id	未実装	Dify を使わないため不要になり、カラムごと落としています
users , user_identities	追加	マルチユーザー識別。同一メールの Google／Outlook を1人に統合
oauth_tokens	追加	ユーザー×プロバイダごとのトークン。期限切れは自動リフレッシュ
oauth_tokens.selected_calendar_ids	追加	空き時間チェックの対象カレンダー（最大3件）
oauth_tokens.write_calendar_ids	追加	新規予定の登録先（最大3件・同時登録）
user_settings	追加	通知設定。配信ジョブがサーバー側で判断するため、端末のlocalStorageだけでは足りない
push_subscriptions	追加	プッシュ通知の宛先。1ユーザーが複数端末を持てる
sent_notifications	追加	二重送信の防止記録。送信前に予約を入れ、重複したら送らない

仕様書に無い、追加した機能

機能	追加した理由
候補枠の仮押さえ	候補日を相手に送ってから返事が来るまでの間に、別の予定で埋まってしまう問題への対処。候補すべてを「[仮]」付きで実カレンダーに入れ、本確定すると残りを自動削除します
1アカウント内の複数カレンダー指定	仕様書は「Google か Outlook か」の粒度しか想定していませんでしたが、実際には1つのGoogleアカウントに仕事用・個人用・共有カレンダーが同居します。参照用と登録先を別々に、各最大3件まで選べるようにしました
重複予定の統合表示	上記で3カレンダーに登録すると予定一覧に同じ予定が3件並び、リマインダーも3回鳴っていました。件名・開始・終了が一致するものを1件にまとめています
未連携カレンダーのボタン無効化	連携していないカレンダーへの登録ボタンが押せてしまい、押すと必ず失敗する状態だったため

仕様にあるが、まだ入っていないもの

項目	仕様書の記述	現状
朝のブリーフィング配信	毎朝7時に Cron が起動しプッシュ通知を送る（UC-1）	実装済み 。ただし Cron サービスは使わず、常時起動しているバックエンド内で5分おきに判定しています（外部Cronは1本あたり月\$1の別課金になるため）。配信時刻はユーザーごとの設定に従います
フラッシュリマインダー	予定のX分前に通知（UC-7）	実装済み 。アプリを閉じていても Web Push で届きます。iPhone は iOS 16.4 以降かつ「ホーム画面に追加」した状態が条件です。アプリを開いている間の画面内フラッシュも従来どおり動きます
オフライン対応	15～30分ごとの背景同期、暫定データバナー、オフライン中の登録キュー（\$8）	いずれも未実装
キャッシュ優先参照	カレンダーAPIより先に Supabase を参照する（\$6-3）	毎回カレンダーAPIを直接見えています。 <code>events</code> テーブルは書き込み時の記録用にとどまります
プロンプトキャッシング	システムプロンプトとツール定義を再利用してコスト削減（\$7-2）	未実装。有効化すればAPI費用を下げられる余地があります
Whisper 音声認識	<code>POST /api/voice</code> （Web Speech 非対応ブラウザ向け）	未実装。ブラウザの Web Speech API のみ対応です
Outlook 連携	Microsoft Graph でGoogleと同等に扱う	コードは実装済みですが、 Azure 側の設定が未完のため未稼働 です

エンドポイントの構成

仕様書 §17 はツールを `POST /api/tools/*` として外部公開する前提でした。これは Dify から呼び戻すためのもので、Dify を外した実装ではツールは HTTP エンドポイントではなく、単なる Python 関数になっています。認証・OAuth もリダイレクト方式に変わりました。

エンドポイント	区分	備考
<code>POST /api/chat</code>	変更	Dify転送ではなく、Claude を直接呼んで SSE で返します
<code>DELETE /api/chat/history</code>	追加	会話リセット時に Claude へ渡す文脈も消すため
<code>POST /api/tools/*</code> (4本)	変更	廃止。プロセス内関数に置き換え
<code>GET /api/auth/{provider}/login</code> <code>GET /api/auth/{provider}/callback</code>	変更	仕様書は <code>POST /api/auth/google</code> でしたが、ブラウザのリダイレクトを伴うため GET 2本構成に
<code>GET /api/events</code>	追加	スケジュール画面用の予定一覧（重複統合済み）
<code>POST /api/events</code>	追加	候補枠の確定登録。会話を介さずボタン操作から直接登録します
<code>POST /api/events/tentative</code> <code>POST /api/events/tentative/release</code>	追加	仮押さえの作成と解除
<code>GET /api/calendars</code> <code>PUT /api/calendars/{provider}/selection</code>	追加	カレンダー一覧の取得と、参照・登録先の選択
<code>PATCH /api/tasks/{id}</code> <code>DELETE /api/tasks/{id}</code>	追加	タスクの編集・削除
<code>GET /api/push/public-key</code> <code>POST /api/push/subscribe</code> <code>POST /api/push/unsubscribe</code> <code>POST /api/push/test</code>	追加	プッシュ通知の購読管理と到達テスト
<code>GET /api/settings</code> <code>PUT /api/settings</code>	追加	通知設定。配信ジョブが参照するため端末だけでなくDBにも保持する

エンドポイント	区分	備考
POST /api/jobs/reminders POST /api/jobs/briefing	追加	配信ジョブの手動起動。共有シークレット必須。通常はアプリ内のループが回すため任意
POST /api/voice	未実装	Whisper 未対応

NEXT

仕様書側で更新したほうがよい箇所

実装が動いている以上、仕様書のほうが現実と食い違っている状態です。外部審査や法人商談で仕様書を出す場面を考えると、少なくとも次の5点は書き換えておく価値があります。

- §6 アーキテクチャ— Dify を前提とした構成図とバージョン変遷表。実装は v3.0 相当に戻っています
- §9・§12・§13 技術スタックとコスト— Dify と Hetzner が不要になり、その分 Render が有料（\$7/月）に変わりました
- 重要な設計制約（付録）— 「トークンをサーバーに保存しない」「メモ判定は毎回 Claude」の2点が、現在の実装と矛盾しています
- §10 認証— マルチユーザー対応（§10-4 として作成済みの原稿）がまだ貼り込まれていません
- §16 テーブル設計— users / user_identities / oauth_tokens / user_settings / push_subscriptions / sent_notifications の6テーブルが未記載です
- §8 オフライン対応— 「15～30分ごとのバックグラウンド同期」は Periodic Background Sync API が前提ですが、これは iOS に存在しません。iPhoneでは原理的に実現できない要件なので、達成可能な形（キャッシュ表示 + 最終更新バナー）へ書き換えが必要です
- UC-1 / UC-7 の配信方式— Cron サービス前提の記述を、アプリ内ループ + Web Push の現行方式に改める。あわせて iOS では「ホーム画面に追加」が受信条件であることを明記すると、テスト時の混乱を防げます

補足

仮押さえ・複数カレンダー登録・重複統合の3機能は、仕様書のどのフェーズにも記載がない純粋な追加です。Phase 1 の機能一覧に足しておくと、テスト結果を投資家や法人に説明する際に話が通りやすくなります。

